

Mounted at /content/drive

Wed Nov 5 08:18:18 2025

文A

```
Installing collected packages: tfty
Successfully installed tfty-6.3.1
Collecting omegaconf==2.0.0
  Downloading omegaconf-2.0.0-py3-none-any.whl.metadata (3.5 kB)
Collecting pytorch-lightning==1.0.8
  Downloading pytorch_lightning-1.0.8-py3-none-any.whl.metadata (26 kB)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages (from omegaconf==2.0.0) (6.0.3)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from omegaconf==2.0.0) (4.15.
Requirement already satisfied: numpy>=1.16.4 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning==1.0.8) (2
Requirement already satisfied: torch>=1.3 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning==1.0.8) (2.8
Requirement already satisfied: future>=0.17.1 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning==1.0.8) (
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning==1.0.8) (4.
Requirement already satisfied: fsspec>=0.8.0 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning==1.0.8) (2
Requirement already satisfied: tensorboard>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from pytorch-lightning==1.0
Requirement already satisfied: absl-py>=0.4 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.2.0->pytorch-l
Requirement already satisfied: grpcio>=1.48.2 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.2.0->pytorch
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.2.0->pytorch
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.2.0->pytorch-ligh
Requirement already satisfied: protobuf<=4.24.0,>=3.19.6 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.2
Requirement already satisfied: setuptools>=41.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.2.0->pyt
Requirement already satisfied: six<1.9 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.2.0->pytorch-lightn
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tens
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard>=2.2.0->pytorch
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->pytorch-lightning==1.
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->pytorch-lightnin
```

```
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->pytorch-lightning==1.
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->pytorch-lightning==1.0.
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->p
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->p
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->py
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->pytor
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->pyto
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->p
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.3->pytorch-lightnin
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tenso
Downloading omegaconf-2.0.0-py3-none-any.whl (33 kB)
Downloading pytorch_lightning-1.0.8-py3-none-any.whl (561 kB)
```

561.4/561.4 kB 36.7 MB/s eta 0:00:00

Installing collected packages: omegaconf, pytorch-lightning

Attempting uninstall: omegaconf

Found existing installation: omegaconf 2.3.0

Uninstalling omegaconf-2.3.0:

```
import PIL
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
#pytorch Library
import torch,os, imageio, pdb, math
import torchvision.transforms as T
import torchvision.transforms.functional as TF
```

!pip install torchvision

```
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.23.0+cu126)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: torch==2.8.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.8.0+cu126)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (4.12.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (12.5.4.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (2.27.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch==2.8.0->torchvision) (3.4.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch==2.8.0->torchvision) (1.13.3)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch==2.8.0->torchvision) (2.1.1)
```

```
import yaml
from omegaconf import OmegaConf
from CLIP import clip
```

```
import warnings
warnings.filterwarnings("ignore")
```

! pip install ftfy

```
Requirement already satisfied: ftfy in /usr/local/lib/python3.12/dist-packages (6.3.1)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from ftfy) (0.2.14)
```

```
def show_from_tensor(tensor):
    img = tensor.clone()
```



```

from taming.models.vqgan import VQModel
import torch
from omegaconf import OmegaConf
import yaml
import torchvision # Added import

def load_config(config_path, display=False):
    config_data = OmegaConf.load(config_path)
    if display:
        print(yaml.dump(OmegaConf.to_container(config_data)))
    return config_data

def load_vqgan(config, chk_path=None):
    model = VQModel(**config.model.params)
    if chk_path is not None:
        state_dict = torch.load(chk_path, map_location="cpu", weights_only=False)["state_dict"] # Added weights_only=False
        missing, unexpected = model.load_state_dict(state_dict, strict=False)
        return model.eval()

def generator(x):
    x = taming_model.post_quant_conv(x)
    x = taming_model.decoder(x)
    return x

taming_config = load_config("./models/vqgan_imagenet_f16_16384/configs/model.yaml", display=True)
taming_model = load_vqgan(taming_config, chk_path="./models/vqgan_imagenet_f16_16384/checkpoints/last.ckpt").to(device)

```

```

model:
  base_learning_rate: 4.5e-06
  params:
    ddconfig:
      attn_resolutions:
        - 16
      ch: 128
      ch_mult:
        - 1
        - 1
        - 2
        - 2
        - 4
      double_z: false
      dropout: 0.0
      in_channels: 3
      num_res_blocks: 2
      out_ch: 3
      resolution: 256
      z_channels: 256
    embed_dim: 256
    lossconfig:
      params:
        codebook_weight: 1.0
        disc_conditional: false
        disc_in_channels: 3
        disc_num_layers: 2
        disc_start: 0
        disc_weight: 0.75
        target: taming.modules.losses.vqperceptual.VQLPIPSWithDiscriminator
      monitor: val/rec_loss
      n_embed: 16384
    target: taming.models.vqgan.VQModel

```

Working with z of shape (1, 256, 16, 16) = 65536 dimensions.

Downloading: "<https://download.pytorch.org/models/vgg16-397923af.pth>" to /root/.cache/torch/hub/checkpoints/vgg16-397923af.p

100%|██████████| 528M/528M [00:05<00:00, 103MB/s]

Downloading vgg_lpips model from <https://heibox.uni-heidelberg.de/f/607503859c864bc1b30b/?dl=1> to taming/modules/autoencoder

8.19kB [00:00, 524kB/s]

loaded pretrained LPIPS loss from taming/modules/autoencoder/lpips/vgg.pth

VQLPIPSWithDiscriminator running with hinge loss.

Declare the values that we are going to optimize

```

class Parameters(torch.nn.Module):
    def __init__(self):
        super(Parameters, self).__init__()
        self.data = .5*torch.randn(batch_size, 256, size1//16, size2//16).cuda() # 1x256x14x15 (225/16, 400/16)
        self.data = torch.nn.Parameter(torch.sin(self.data))

    def forward(self):
        return self.data

def init_params():
    params=Parameters().cuda()
    optimizer = torch.optim.AdamW([{'params':[params.data], 'lr': learning_rate}], weight_decay=wd)
    return params, optimizer

```

```

### Encoding prompts and a few more things

import torchvision # Added import here

normalize = torchvision.transforms.Normalize((0.48145466, 0.4578275, 0.40821073), (0.26862954, 0.26130258, 0.27577711))

def encodeText(text):
    t=clip.tokenize(text).cuda()
    t=clipmodel.encode_text(t).detach().clone()
    return t

def createEncodings(include, exclude, extras):
    include_enc=[]
    for text in include:
        include_enc.append(encodeText(text))
    exclude_enc=encodeText(exclude) if exclude != '' else 0
    extras_enc=encodeText(extras) if extras !='' else 0

    return include_enc, exclude_enc, extras_enc

augTransform = torch.nn.Sequential(
    torchvision.transforms.RandomHorizontalFlip(),
    torchvision.transforms.RandomAffine(30, (.2, .2), fill=0)
).cuda()

# Ensure taming_model is defined before this cell is executed.
# It is defined in the cell containing the following code block:
# from taming.models.vqgan import VQModel
# def load_config(...): ...
# def load_vqgan(...): ...
# def generator(...): ...
# taming_config = load_config(...)
# taming_model = load_vqgan(...)

Params, optimizer = init_params()

with torch.no_grad():
    print(Params().shape)
    img= norm_data(generator(Params()).cpu()) # 1 x 3 x 224 x 400 [225 x 400]
    print("img dimensions: ",img.shape)
    show_from_tensor(img[0])

```

```

### create crops

def create_crops(img, num_crops=32):
    p=size1//2
    img = torch.nn.functional.pad(img, (p,p,p,p), mode='constant', value=0) # 1 x 3 x 448 x 624 (adding 112*2 on all sides to

```

```

img = augTransform(img) #RandomHorizontalFlip and RandomAffine

crop_set = []
for ch in range(num_crops):
    gap1= int(torch.normal(1.2, .3, ())).clip(.43, 1.9) * size1
    offsetx = torch.randint(0, int(size1*2-gap1),())
    offsety = torch.randint(0, int(size1*2-gap1),())

    crop=img[:, :,offsetx:offsetx+gap1, offsety:offsety+gap1]

    crop = torch.nn.functional.interpolate(crop,(224,224), mode='bilinear', align_corners=True)
    crop_set.append(crop)

img_crops=torch.cat(crop_set,0) ## 30 x 3 x 224 x 224

randnormal = torch.randn_like(img_crops, requires_grad=False)
num_rands=0
randstotal=torch.rand((img_crops.shape[0],1,1,1)).cuda() #32

for ns in range(num_rands):
    randstotal*=torch.rand((img_crops.shape[0],1,1,1)).cuda()

img_crops = img_crops + noise_factor*randstotal*randnormal

return img_crops

```

Show current state of generation

```

def showme(Params, show_crop):
    with torch.no_grad():
        generated = generator(Params())

        if (show_crop):
            print("Augmented cropped example")
            aug_gen = generated.float() # 1 x 3 x 224 x 400
            aug_gen = create_crops(aug_gen, num_crops=1)
            aug_gen_norm = norm_data(aug_gen[0])
            show_from_tensor(aug_gen_norm)

        print("Generation")
        latest_gen=norm_data(generated.cpu()) # 1 x 3 x 224 x 400
        show_from_tensor(latest_gen[0])

    return (latest_gen[0])

```

Optimization process

```

def optimize_result(Params, prompt):
    alpha=1 ## the importance of the include encodings
    beta=.5 ## the importance of the exclude encodings

    ## image encoding
    out = generator(Params())
    out = norm_data(out)
    out = create_crops(out)
    out = normalize(out) # 30 x 3 x 224 x 224
    image_enc=clipmodel.encode_image(out) ## 30 x 512

    ## text encoding w1 and w2
    final_enc = w1*prompt + w1*extras_enc # prompt and extras_enc : 1 x 512
    final_text_include_enc = final_enc / final_enc.norm(dim=-1, keepdim=True) # 1 x 512
    final_text_exclude_enc = exclude_enc

    ## calculate the loss
    main_loss = torch.cosine_similarity(final_text_include_enc, image_enc, -1) # 30
    penalize_loss = torch.cosine_similarity(final_text_exclude_enc, image_enc, -1) # 30

    final_loss = -alpha*main_loss + beta*penalize_loss

    return final_loss

def optimize(Params, optimizer, prompt):
    loss = optimize_result(Params, prompt).mean()
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss

```

```
### training loop

def training_loop(Params, optimizer, show_crop=False):
    res_img=[]
    res_z=[]

    for prompt in include_enc:
        iteration=0
        Params, optimizer = init_params() # 1 x 256 x 14 x 25 (225/16, 400/16)

        for it in range(total_iter):
            loss = optimize(Params, optimizer, prompt)

            if iteration>=80 and iteration%show_step == 0:
                new_img = showme(Params, show_crop)
                res_img.append(new_img)
                res_z.append(Params()) # 1 x 256 x 14 x 25
                print("loss:", loss.item(), "\niteration:",iteration)

            iteration+=1
        torch.cuda.empty_cache()
    return res_img, res_z
```

```
torch.cuda.empty_cache()
include=['Green tree in the forest']
exclude='watermark'
extras = ""
w1=1
w2=1
noise_factor= .22
total_iter=110
show_step=10 # set this to see the result every 10 iterations beyond iteration 80
include_enc, exclude_enc, extras_enc = createEncodings(include, exclude, extras)
res_img, res_z=training_loop(Params, optimizer, show_crop=True)
```

```
import torch # Added import
import torchvision # Added import here
from CLIP import clip # Added import here

### CLIP MODEL ###
clipmodel, _ = clip.load('ViT-B/32', jit=False)
clipmodel.eval()

def encodeText(text):
    t=clip.tokenize(text).cuda()
    t=clipmodel.encode_text(t).detach().clone()
    return t

def createEncodings(include, exclude, extras):
    include_enc=[]
    for text in include:
        include_enc.append(encodeText(text))
    exclude_enc=encodeText(exclude) if exclude != '' else 0
    extras_enc=encodeText(extras) if extras !='' else 0

    return include_enc, exclude_enc, extras_enc

torch.cuda.empty_cache()
include = ['sunset over mountains', 'river reflection', 'birds flying']
exclude='watermark'
extras = ""
w1=1
w2=1
noise_factor= .22
total_iter=110
show_step=10 # set this to see the result every 10 interations beyond iteration 80
include_enc, exclude_enc, extras_enc = createEncodings(include, exclude, extras)
res_img, res_z = training_loop(Params, optimizer, show_crop=True)
```



```
def interpolate(res_z_list, duration_list):
    gen_img_list=[]
    fps = 25

    for idx, (z, duration) in enumerate(zip(res_z_list, duration_list)):
        num_steps = int(duration*fps)
        z1=z
        z2=res_z_list[(idx+1)%len(res_z_list)] # 1 x 256 x 14 x 25 (225/16, 400/16)

        for step in range(num_steps):
            alpha = math.sin(1.5*step/num_steps)**6
            z_new = alpha * z2 + (1-alpha) * z1

            new_gen=norm_data(generator(z_new).cpu())[0] ## 3 x 224 x 400
            new_img=T.ToPILImage(mode='RGB')(new_gen)
            gen_img_list.append(new_img)

    return gen_img_list

durations=[5,5,5,5,5,5]
interp_result_img_list = interpolate(res_z, durations)
```

```
## create a video
out_video_path=f"../video.mp4"
writer = imageio.get_writer(out_video_path, fps=25)
for pil_img in interp_result_img_list:
    img = np.array(pil_img, dtype=np.uint8)
    writer.append_data(img)

writer.close()
```

```
from IPython.display import HTML
from base64 import b64encode

mp4 = open('../video.mp4', 'rb').read()
data="data:video/mp4;base64,"+b64encode(mp4).decode()
HTML("""<video width=800 controls><source src="%s" type="video/mp4"></video>""" % data)
```

Start coding or [generate](#) with AI.

